



smelt-ml: A Pure-Rust Machine Learning Framework with Cache-Optimized Gradient Boosting Competitive with C++ Implementations

Francisco Parra

Universidad de Santiago de Chile

Abstract

We present **smelt-ml**, an open-source machine learning framework implemented entirely in Rust that provides 27 supervised learning algorithms, unsupervised clustering, causal inference, survival analysis, and spatial machine learning capabilities. The framework's gradient boosting implementations (**XGBoost**, **LightGBM**, **CatBoost**) employ a cache-optimized column-major histogram accumulation strategy that achieves performance competitive with — and in many cases exceeding — the official C++ implementations. On standardized benchmarks with 20 features and up to 10,000 samples, our **XGBoost** is 1.1–2.6× faster, **LightGBM** 1.2–1.4× faster, and **CatBoost** 1.3–4.2× faster than their respective C++ counterparts. The framework is published on <https://crates.io/> as **smelt-ml** and provides a composable, trait-based architecture inspired by **mlr3** (Lang, Binder, Richter, Schratz, Pfisterer, Coors, Au, Casalicchio, Kotthoff, and Bischl 2019) and **scikit-learn** (Pedregosa, Varoquaux, Gramfort, Michel, Thirion, Grisel, Blondel, Prettenhofer, Weiss, Dubourg, Vanderplas, Passos, Cournapeau, Brucher, Perrot, and Duchesnay 2011).

Keywords: machine learning, gradient boosting, Rust, cache optimization, **XGBoost**, **LightGBM**, **CatBoost**, software.

1. Introduction

Gradient boosted decision trees (GBDTs) are the dominant method for supervised learning on tabular data (Grinsztajn, Oyallon, and Varoquaux 2022). The three most widely used implementations — **XGBoost** (Chen and Guestrin 2016), **LightGBM** (Ke, Meng, Finley, Wang, Chen, Ma, Ye, and Liu 2017), and **CatBoost** (Prokhorenkova, Gusev, Vorobev, Dorogush, and Gulin 2018) — are implemented in C++ and have been optimized over many years. These

libraries serve as the de facto baselines in the majority of applied machine learning studies.

Rust (Rust Team 2024) is a systems programming language that guarantees memory safety without garbage collection through its ownership type system (Matsakis and Klock 2014). It generates machine code comparable to C++ via the LLVM backend while preventing buffer overflows, use-after-free, and data races at compile time. Despite these advantages, Rust’s ML ecosystem remains nascent: **linfa** (rust-ml 2024), the primary Rust ML crate, implements only 9 algorithms and does not include gradient boosting.

In this paper, we present **smelt-ml**, a comprehensive ML framework in Rust that fills this gap. Our main contributions are:

1. A **complete ML framework** with 27 supervised learners, clustering, survival analysis, causal inference, and spatial ML — the most comprehensive ML framework in Rust.
2. **Cache-optimized gradient boosting** using column-major bin storage with u8 packing, achieving performance that matches or exceeds the official C++ implementations.
3. **Novel algorithms** not available in competing frameworks: Geographical-XGBoost (Grekousis 2025), Causal Forest (Wager and Athey 2018), Conformal Prediction (Romano, Patterson, and Candès 2019), Oblique Forest (Tomita, Browne, Shen, Chung, Patsolic, Falk, Priebe, Yim, Burns, Maggioni, and Vogelstein 2020), and streaming Hoeffding Trees.
4. **Validation on real datasets** showing accuracy comparable to **scikit-learn** across standard benchmarks.

2. Software architecture

2.1. Design principles

smelt-ml follows a trait-based architecture inspired by **mlr3**:

```
Data (CSV) -> Task -> Pipeline(Transformers -> Learner)
                        -> Prediction -> Measure
```

The core abstractions are defined as Rust traits:

- **Learner**: Algorithm that trains on a **Task** and produces a **TrainedModel**. Requires **Send + Sync** for thread safety.
- **TrainedModel**: Fitted model that predicts on new data.
- **Measure**: Evaluation metric (Accuracy, F1, RMSE, R^2 , AUC-ROC, etc.).
- **Transformer**: Preprocessing step composable via **Pipeline**.
- **Resample**: Data splitting strategy.

Category	Algorithm	Classif.	Regress.
Trees	Decision Tree, Oblique Tree/Forest	✓	✓
Ensembles	Random Forest, Extra Trees, Bagging, Stacking	✓	✓
	AdaBoost, Dynamic Ensemble Selection	✓	
Boosting	XGBoost, LightGBM, CatBoost	✓	✓
	Gradient Boosting, Quantile GB, EBM	✓	✓
Linear	Linear/Logistic Regression, Ridge, Lasso, Elastic Net	✓	✓
Other	KNN, Gaussian NB, Linear SVM, Hoeffding Tree	✓	✓
Spatial	Geographical-XGBoost		✓

Table 1: Supervised learners in **smelt-ml**.

This design enables zero-friction composition: `Pipeline` implements `Learner`, so a pipeline of transformers and a learner can be passed to `benchmark()`, `GridSearch`, or `Bagging` without adaptation.

2.2. Quick start

A minimal classification example:

```
rust> use smelt_ml::prelude::*;
rust> use ndarray::array;
rust>
rust> let features = array![[0.0, 0.0], [1.0, 1.0],
+   [0.0, 1.0], [1.0, 0.0]];
rust> let target = vec![0, 1, 1, 0];
rust> let task = ClassificationTask::new("xor", features, target)
+   .unwrap();
rust>
rust> let mut xgb = XGBoost::new()
+   .with_n_estimators(50)
+   .with_max_depth(3);
rust> let model = xgb.train_classif(&task).unwrap();
rust> let pred = model.predict(task.features()).unwrap();
```

2.3. Implemented algorithms

Table 1 summarizes the 27 supervised learners. Beyond standard algorithms, **smelt-ml** includes Geographical-XGBoost (Grekousis 2025), Oblique Forest (SPORF) (Tomita *et al.* 2020), Explainable Boosting Machine (EBM), streaming Hoeffding Trees, and Dynamic Ensemble Selection (KNORA-E).

Additional modules provide: K-Means, DBSCAN, and Isolation Forest (clustering/anomaly); Random Survival Forest with C-index (Ishwaran, Kogalur, Blackstone, and Lauer 2008) (survival analysis); Causal Forest with honest splitting (Athey and Imbens 2016; Wager and Athey 2018) (causal inference); Classifier Chains and Regressor Chains (multi-label/output);

Quantile Regression Forest; Conformal Prediction with CQR (Romano *et al.* 2019); and SHAP-based feature importance.

3. Cache-optimized gradient boosting

3.1. The histogram bottleneck

Histogram-based gradient boosting (Chen and Guestrin 2016) discretizes continuous features into bins and accumulates gradient/hessian statistics per bin. The key computational kernel is:

$$\text{hist}[b] += g_i, \quad b = \text{bin_index}[i][\text{feature}] \quad (1)$$

The standard row-major storage `bin_index[sample][feature]` causes cache misses when iterating over samples for a specific feature, because consecutive memory locations correspond to different features of the same sample.

3.2. Column-major storage with u8 packing

Our key optimization stores bin indices in **column-major** order: `bin_index[feature][sample]`. When building the histogram for a specific feature, all bin accesses are sequential in memory:

```
rust> // Column-major: sequential access (cache-friendly)
rust> for &idx in node_indices {
+     let bin = bins.get_bin(feature, idx);
+     bin_g[bin] += gradients[idx];
+     bin_h[bin] += hessians[idx];
+ }
```

Combined with **u8 packing** (254 real bins + 1 NaN sentinel fit in 1 byte vs. 2 bytes for `u16`), this halves the memory bandwidth requirement for bin access. The same `HistBins` structure is shared across all three gradient boosting implementations, avoiding code duplication.

3.3. Additional optimizations

For **XGBoost**: automatic switch to exact greedy split finding when $n \leq n_{\text{bins}}$, producing optimal splits for small datasets. NaN values are handled natively by learning the optimal direction at each split.

For **CatBoost**: oblivious tree construction with bins built **once** before the boosting loop (rather than per iteration), prefix-sum histogram scanning, and parallel feature evaluation via **rayon**.

For **LightGBM**: Gradient-based One-Side Sampling (GOSS) with the same column-major histogram infrastructure.

4. Performance evaluation

n	XGBoost			LightGBM			CatBoost		
	C++	Rust	$\times$	C++	Rust	$\times$	C++	Rust	$\times$
500	72	28	2.6	49	53	0.9	253	60	4.2
1,000	90	48	1.9	104	88	1.2	254	98	2.6
5,000	191	139	1.4	174	186	0.9	324	209	1.6
10,000	274	253	1.1	209	243	0.9	445	350	1.3

Table 2: Classification training time (ms). Speedup > 1 means **smelt-ml** is faster. All values single-threaded.

Dataset	Learner	smelt-ml	scikit-learn	Δ
Iris	Decision Tree	0.953	0.953	0.000
	Random Forest	0.953	0.967	-0.014
	XGBoost	0.953	0.953	0.000
Wine	Random Forest	0.983	0.972	+0.011
	Logistic Reg.	0.989	0.961	+0.028
	XGBoost	0.971	0.944	+0.027
Breast Ca.	Random Forest	0.958	0.956	+0.002
	XGBoost	0.970	0.974	-0.004

Table 3: 5-fold CV accuracy on real datasets.

4.1. Experimental setup

Benchmarks use synthetic datasets from **scikit-learn**’s `make_classification()` and `make_regression()` (20 features, 10 informative, n from 100 to 10,000). Configuration: 100 trees, max depth 6, learning rate 0.3 (**XGBoost**/**CatBoost**) or 0.1 (**LightGBM**). Single-threaded execution. Rust compiled with `target-cpu=native` (release mode). Official libraries via Python API with `n_jobs=1`.

4.2. Training time

Table 2 shows classification training times.

smelt-ml’s **XGBoost** is consistently faster (1.1–2.6 $\times$), **CatBoost** is substantially faster (1.3–4.2 $\times$), and **LightGBM** is competitive (0.9–1.2 $\times$). Similar patterns hold for regression tasks.

4.3. Accuracy validation

Table 3 validates correctness against **scikit-learn** on standard datasets.

All comparisons are within $\pm 3\%$ of **scikit-learn**.

5. Unique features

5.1. Geographical-XGBoost

Implementation of [Grekousis \(2025\)](#): bi-square spatial kernel weights, local **XGBoost** models per spatial unit, ensemble of global and local predictions with adaptive α weight. First implementation outside the original Python library.

5.2. Causal Forest

Honest splitting following [Athey and Imbens \(2016\)](#) and [Wager and Athey \(2018\)](#). Estimates conditional average treatment effects (CATE) $\tau(x) = E[Y(1) - Y(0)|X = x]$ with 95% confidence intervals. First implementation in a Rust ML framework.

5.3. Conformal prediction

Split conformal prediction for regression and classification with conformalized quantile regression (CQR; [Romano et al. 2019](#)). Provides distribution-free prediction intervals with guaranteed coverage.

6. Summary and discussion

We have demonstrated that a pure-Rust implementation of gradient boosted trees can match or exceed the performance of highly optimized C++ libraries through careful attention to memory layout and cache utilization. The performance advantage stems from Rust’s ownership model, which guarantees no aliasing (`&mut` is unique) and enables LLVM to apply more aggressive auto-vectorization than is possible with C++, where pointer aliasing must be assumed unless explicitly annotated.

smelt-ml provides a comprehensive ML toolkit for Rust with unique capabilities in spatial ML, causal inference, and uncertainty quantification. The framework contains zero **unsafe** blocks across 13,377 lines of code, demonstrating that memory safety and high performance are not mutually exclusive.

Limitations include the performance gap on datasets beyond 10,000 samples (where C++ libraries employ SIMD intrinsics and GPU backends), the absence of GPU support, and the simplified ordered boosting in **CatBoost**.

The software is available at <https://crates.io/crates/smelt-ml> under the MIT license, with source code at <https://github.com/franciscoparrao/smelt>.

Computational details

The results in this paper were obtained using Rust 1.89 with the **ndarray** 0.16, **rayon** 1.10, and **serde** 1.0 crates. Official C++ libraries were accessed via Python 3.12 with **xgboost** 3.1, **lightgbm** 4.6, and **catboost** 1.2. **scikit-learn** 1.8 was used for accuracy validation. Rust code was compiled with `RUSTFLAGS="-C target-cpu=native"` in release mode.

Acknowledgments

[To be completed]

References

- Athey S, Imbens G (2016). “Recursive Partitioning for Heterogeneous Causal Effects.” *Proceedings of the National Academy of Sciences*, **113**(27), 7353–7360. doi:[10.1073/pnas.1510489113](https://doi.org/10.1073/pnas.1510489113).
- Chen T, Guestrin C (2016). “**XGBoost**: A Scalable Tree Boosting System.” In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. doi:[10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- Grekousis G (2025). “Geographical-**XGBoost**: A New Ensemble Model for Spatially Local Regression Based on Gradient-Boosted Trees.” *Journal of Geographical Systems*, **27**(2), 169–195. doi:[10.1007/s10109-025-00465-4](https://doi.org/10.1007/s10109-025-00465-4).
- Grinsztajn L, Oyallon E, Varoquaux G (2022). “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?” In *Advances in Neural Information Processing Systems* 35.
- Ishwaran H, Kogalur UB, Blackstone EH, Lauer MS (2008). “Random Survival Forests.” *Annals of Applied Statistics*, **2**(3), 841–860. doi:[10.1214/08-A0AS169](https://doi.org/10.1214/08-A0AS169).
- Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, Ye Q, Liu TY (2017). “**LightGBM**: A Highly Efficient Gradient Boosting Decision Tree.” In *Advances in Neural Information Processing Systems* 30.
- Lang M, Binder M, Richter J, Schratz P, Pfisterer F, Coors S, Au Q, Casalicchio G, Kothhoff L, Bischl B (2019). “**mlr3**: A Modern Object-Oriented Machine Learning Framework in R.” *Journal of Open Source Software*, **4**(44), 1903. doi:[10.21105/joss.01903](https://doi.org/10.21105/joss.01903).
- Matsakis ND, Klock FS (2014). “The Rust Language.” *Ada Letters*, **34**(3), 103–104. doi:[10.1145/2692956.2663188](https://doi.org/10.1145/2692956.2663188).
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay É (2011). “**Scikit-learn**: Machine Learning in Python.” *Journal of Machine Learning Research*, **12**, 2825–2830.
- Prokhorenkova L, Gusev G, Vorobev A, Dorogush AV, Gulin A (2018). “**CatBoost**: Unbiased Boosting with Categorical Features.” In *Advances in Neural Information Processing Systems* 31.
- Romano Y, Patterson E, Candès E (2019). “Conformalized Quantile Regression.” In *Advances in Neural Information Processing Systems* 32.
- rust-ml (2024). *linfa: A Rust Machine Learning Framework*. Version 0.7, URL <https://github.com/rust-ml/linfa>.
- Rust Team (2024). *The Rust Programming Language*. Version 1.89, URL <https://www.rust-lang.org/>.

Tomita TM, Browne J, Shen C, Chung J, Patsolic JL, Falk B, Priebe CE, Yim J, Burns R, Maggioni M, Vogelstein JT (2020). “Sparse Projection Oblique Randomer Forests.” *Journal of Machine Learning Research*, **21**(104), 1–39.

Wager S, Athey S (2018). “Estimation and Inference of Heterogeneous Treatment Effects Using Random Forests.” *Journal of the American Statistical Association*, **113**(523), 1228–1242. doi:10.1080/01621459.2017.1319839.

Affiliation:

Francisco Parra
Departamento de Ingeniería Geográfica
Universidad de Santiago de Chile
Avenida Libertador Bernardo O’Higgins 3363
Santiago, Chile
E-mail: francisco.parra.o@usach.cl